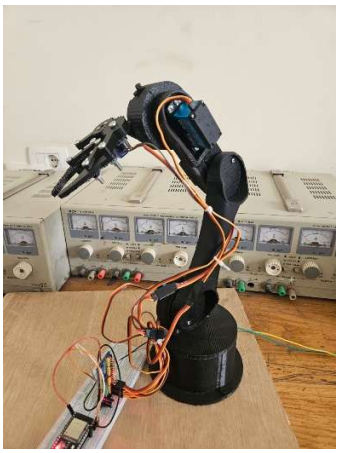
# Wireless Master-Slave 5 DoF Robotic Arm

MET 467: Mechatronics Project






**Team Members**

Mohamed Alaa-eldeen Madbouly

Ahmed Mohamed Ahmed Abdel-Aziz

Rami Awad Issa

Abdul-Rahman Mohamed Abdo Fawzy

Beshoy Medhat Monir

**Supervisor**

**Assoc. Prof. Dr. Shuaiby Mohamed**

Mechatronics and Robotics Engineering Program

Faculty of Engineering

Assiut University


2025-2026

# Abstract

This project presents the design and implementation of a wireless master–slave robotic system consisting of a 5-degree-of-freedom (5-DOF) slave robot arm controlled by a corresponding master arm. The system was developed from scratch to explore practical concepts in mechatronics, including mechanical design, embedded systems, and wireless communication.

The main idea is to capture the motion of the master arm and transmit it wirelessly to the slave arm, which replicates the movement accurately. The system operates in two primary modes: a real-time mode, where the motion is transmitted instantly with minimal delay for direct teleoperation, and a record-and-execute mode, where up to four poses can be stored and replayed automatically. In addition, a bonus mode is proposed using the MoveIt package in ROS2 to enable advanced motion planning and control of the slave arm. The project successfully demonstrates reliable wireless control, accurate motion replication, and flexible operation modes, highlighting its potential for applications in automation, remote manipulation, and robotic research.

# Table of Contents

Abstract ..... I
List of Figures and Tables ..... III
1. Introduction ..... 1
2. Problem Statement ..... 2
3. Project Objectives ..... 3
4. Mechatronic System Design ..... 4
4.1 Slave Arm Mechanical Design ..... 4
4.1.1 Robot Arm Structure ..... 4
4.1.2 Degrees of Freedom (DOF) ..... 5
4.1.3 Link Lengths, Joints, and Materials ..... 5
4.1.4 Assembly Process ..... 6
4.1.5 End Effector Changing Mechanism ..... 6
4.1.6 End Effectors: Gripper and Ladle ..... 7
4.1.7 Design Justification ..... 8
4.2 Master Arm Mechanical Design ..... 9
4.2.1 Potentiometer Integration ..... 9
4.2.2 Angle Calibration ..... 10
4.2.3 Fixation and Stability ..... 10
4.2.4 End Effector Mechanism ..... 10
4.3 Electrical and Electronics Design ..... 11
4.3.1 Actuators ..... 12
4.3.2 Battery and Power Consumption ..... 13
5. Control Strategy ..... 15
5.1 Control System / Software ..... 15
5.1.1 Master Arm Control System ..... 15
5.1.2 Slave Arm Control System ..... 18

5.2 Communication Method....20
5.3 Bonus Work: Third Mode (ROS2 and Motion Planning Integration)....21
5.4 Robot Arm Control System Using ROS 2 & MoveIt 2....23
5.4.1 Layer 1: Software Configuration & Launch Infrastructure....23
5.4.2 Layer 2: Trajectory Execution & Hardware Control....23
6. Conclusion....25
Appendices....26

List of Figures

Figure 1: Slave arm ..... 4
Figure 2: Twist Lock Mechanism ..... 7
Figure 3: Gripper with twist lock end ..... 7
Figure 4: Ladle end-effector ..... 8
Figure 5: Ladle end-effector with twist lock end ..... 8
Figure 6: Master Robot Arm ..... 9
Figure 7: Potentiometer fixation ..... 10
Figure 8: Potentionmeter fixation from back ..... 10
Figure 9: Hardware diagram and connections ..... 11
Figure 10: FT7135M Metal Gear Servo (35 kg.cm) ..... 13
Figure 11: SG90 Micro Servo 180 Degree ..... 13
Figure 12: mode 1 algorithm flowchart ..... 16
Figure 13: record algorithm flowchart ..... 17
Figure 14: Mode 3 flowchart diagram ..... 18
Figure 15: Slave arm flowchart ..... 19
Figure 16: data format ..... 20
Figure 17: Rviz & Moveit planning and executing ..... 22
Figure 18: configuration diagram of mode 3 ..... 24
Figure 19: control diagram of mode 3 ..... 24

# 1. Introduction

Robotic arms play a fundamental role in the field of mechatronics due to their wide range of applications in industrial automation, medical systems, hazardous environment handling, and precision tasks. They combine mechanical design, electronics, and intelligent control to perform complex movements with high accuracy and repeatability. As technology advances, the need for flexible and remotely operated robotic systems has become increasingly important, especially in environments that are dangerous or inaccessible to humans.

The purpose of this project is to design and build a wireless master–slave robotic arm system starting from scratch, integrating both hardware and software components. The system consists of a master arm used by the operator to generate motion inputs, and a 5-degree-of-freedom (5-DOF) slave arm that replicates these movements remotely through wireless communication. This approach introduces the concept of master-slave control, where one system (the master) directly controls the behavior of another system (the slave) in real time.

The main problem addressed in this project is enabling safe and efficient control of a robotic arm in areas that are difficult or dangerous for humans to access. Additionally, the system is designed with flexibility in mind, allowing the end effector to be changed depending on the required task, which increases the range of possible applications.

The system operates in two primary modes. In the real-time mode, the motion of the master arm is transmitted instantly to the slave arm, allowing direct and continuous control. In the record-and-execute mode, the user can store up to four different poses, which the slave arm can later reproduce automatically. These modes provide both interactive control and task automation, making the system versatile for different use cases.

To further enhance the system capabilities, a third mode is proposed as a bonus feature using the MoveIt package within ROS 2. The motivation behind this mode is to move beyond manual control and introduce intelligent motion planning and autonomous operation.

# 2. Problem Statement

The use of robotic manipulation or remote control has grown in industrial automation, hazardous environments, medical systems and research systems. Many of the existing robotic arm systems, however, are costly, complex and need to be wired or involve high-end industrial hardware, making them difficult to use in low-cost education and research applications.

Moreover, the control system for robotic arms must be reliable and responsive, be able to accurately reproduce the human motion, and should possess minimal delay and stable operation in operating in dangerous and hard-to-reach environments. Manual manipulation techniques can sometimes put the operators at risk of injury and make operations less efficient.

One of the difficulties is that many robotic systems are rigid and end effectors are not able to change their function, like object gripping, or material handling, depending on the task. In addition, there are low cost robotic arms that are not equipped with intelligent features like motion recording, autonomous execution and advanced motion planning integration.

Hence, a low cost wireless master–slave robotic arm system is needed to accurately replicate human motion in real time, enable automated task execution, and enable flexible end-effector interchangeability. The system should also be highly functional and robust enough to enable seamless wireless communication, smooth robotic control, and compatibility with current robotics systems like ROS2 and MoveIt to enable future smart and semi-autonomous operation.

# 3. Project Objectives

The main objective of this project is to design and implement a complete wireless master–slave robotic arm system that demonstrates the integration of mechanical design, electronics, and intelligent control. The specific objectives of the project are as follows:

- To design and build a 5-degree-of-freedom (5-DOF) slave robotic arm capable of performing precise and smooth movements.
- To develop a master robotic arm that serves as an input device to capture the operator's motion accurately.
- To design a modular and interchangeable end effector mechanism, allowing different tools (such as grippers or specialized attachments) to be easily mounted based on the required task.
- To establish reliable wireless communication between the master and slave systems for remote operation.
- To implement a real-time control mode that enables instant transmission of motion from the master arm to the slave arm with minimal delay.
- To implement a record-and-execute mode that allows storing up to four different poses and replaying them automatically.
- Integrate the robotic arm with ROS2 and MoveIt for motion planning and semi-autonomous operation.

Overall, the system demonstrates a complete mechatronic solution that combines mechanical design, wireless communication, and control algorithms to achieve flexible and efficient robotic manipulation.

# 4. Mechatronic System Design

The proposed wireless master–slave robotic arm system is designed using ESP32 microcontrollers as the central control units. The system integrates mechanical structure, sensing elements, wireless communication, and electronic control to achieve accurate real-time robotic motion replication and automated manipulation tasks in an efficient and flexible manner. The master arm uses potentiometers to capture the operator's joint movements, while the slave arm reproduces these movements using servo motors controlled through a PCA9685 driver module. Wireless communication between both systems is implemented using the ESP-NOW protocol to ensure fast and reliable low-latency data transmission. In addition, the robotic arm features a modular mechanical design with interchangeable end effectors and supports advanced robotics integration through ROS2 and MoveIt for motion planning and semi-autonomous operation.

## 4.1 Slave Arm Mechanical Design

### *4.1.1 Robot Arm Structure*



*Figure 1: Slave arm*

The slave robotic arm consists of a serial chain structure made up of multiple rigid links connected through rotational joints. The design includes a base rotation joint,

followed by shoulder, elbow, and wrist joints, allowing the arm to reach a wide range of positions and orientations. This serial configuration was selected due to its simplicity, ease of control, and suitability for pick-and-place and teleoperation tasks.

### *4.1.2 Degrees of Freedom (DOF)*

The robot arm has five degrees of freedom, which provide sufficient flexibility for positioning and orienting the end effector. The DOFs include:

- Base rotation
- Shoulder movement
- Elbow movement
- Wrist pitch
- Wrist orientation

This configuration allows the arm to reach target positions in 3D space while maintaining control over the end-effector orientation

### *4.1.3 Link Lengths, Joints, and Materials*

The arm is composed of multiple links with carefully selected lengths to balance reach and stability. Longer links increase workspace but may reduce rigidity (and need more torque from motors), so a compromise was made to ensure both adequate reach and structural strength.

All joints are revolute joints driven by motors, enabling smooth rotational motion. The structure is built using lightweight yet strong materials (PLA+ 3D-printed components), which reduce the load on the motors while maintaining durability.

To determine the required motor specifications for the robotic arm joints, a torque analysis was performed based on the mechanical structure and load distribution of the arm. The goal of this calculation is to estimate the **maximum torque at the base joint**, which is the most critical joint since it carries the load of all links and the payload.

**Torque Equation:**

The total required torque $T$ is calculated as the sum of torques generated by each link's weight, motor weights, and the external load. The equation used is:

$$T = \left(w_1 \cdot \frac{l_1}{2}\right) + (m_1 \cdot l_1) + \left(w_2 \cdot \left(l_1 + \frac{l_2}{2}\right)\right) + \left(m_2 \cdot (l_1 + l_2)\right) + \left(w_3 \cdot \left(l_1 + l_2 + \frac{l_3}{2}\right)\right) + \left((m_3 + \text{load}) \cdot (l_1 + l_2 + l_3)\right)$$

**Where:**

- $l_1, l_2, l_3$: lengths of links (cm)
- $w_1, w_2, w_3$: weights of links (kg)
- $m_1, m_2, m_3$: weights of motors mounted on each link (kg)
- **load**: payload at the end effector (kg)
- $T$: total torque (kg·cm)

**Given:**

- $l_1 = 15, l_2 = 11, l_3 = 9\text{cm}$
- $w_1 = 0.15, w_2 = 0.1, w_3 = 0.1\text{kg}$
- $m_1 = m_2 = 0.06, m_3 = 0.16\text{kg}$
- Load = 0.5 kg

**Final Answer"**

$$\boxed{T = 29.66\ \text{kg.cm}}$$

then, the suitable motor is 35 kg.cm (FT7135)

### *4.1.4 Assembly Process*

The robotic arm was assembled step by step, starting from the base and progressing upward through each joint. Each motor was mounted securely to ensure proper alignment and minimize vibration. The links were connected using brackets and fasteners, and special attention was given to maintaining joint stability and reducing mechanical play. Proper cable management was also considered to avoid interference with motion.

### *4.1.5 End Effector Changing Mechanism*

The slave arm uses a twist-lock (bayonet) mechanism to allow quick and secure changing of the end effector, such as a gripper or ladle. The tool is inserted into the wrist mount and locked with a short rotational motion (quarter turn), ensuring proper alignment and firm attachment.

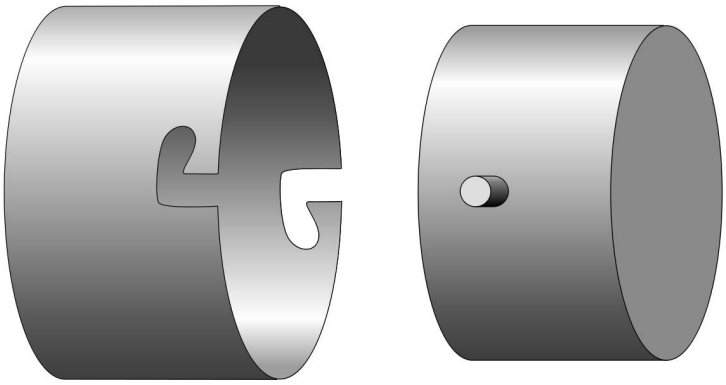


*Figure 2: Twist Lock Mechanism*

This mechanism provides a good balance between speed and stability, allowing fast tool replacement without compromising performance during operation. It also uses a standardized interface, enabling multiple end effectors to be used with the same robotic arm.

### *4.1.6 End Effectors: Gripper and Ladle*

Two interchangeable end effectors were developed for the robotic arm: a gripper and a ladle, each designed for a specific type of task.

The gripper is used for picking and holding solid objects. It is designed with two fingers that open and close to grasp items securely. This makes it suitable for pick-and-place operations and general object manipulation.

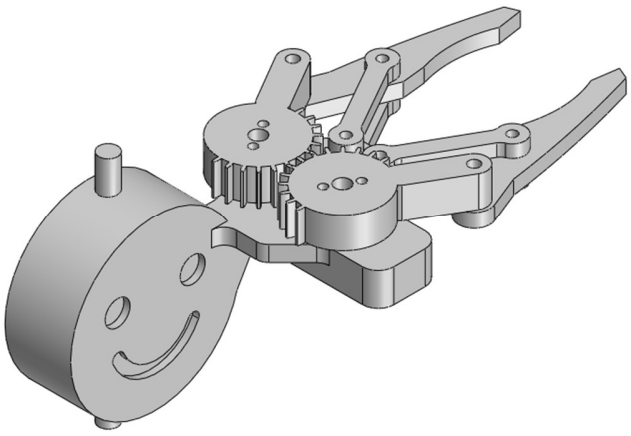


*Figure 3: Gripper with twist lock end*

The **ladle** is designed for handling materials such as liquids or small particles. Instead of gripping, it allows the robot to scoop and transfer materials, which expands the functionality of the system beyond standard object handling.

Both end effectors are compatible with the twist-lock mechanism, allowing quick switching between them depending on the task r equirements.

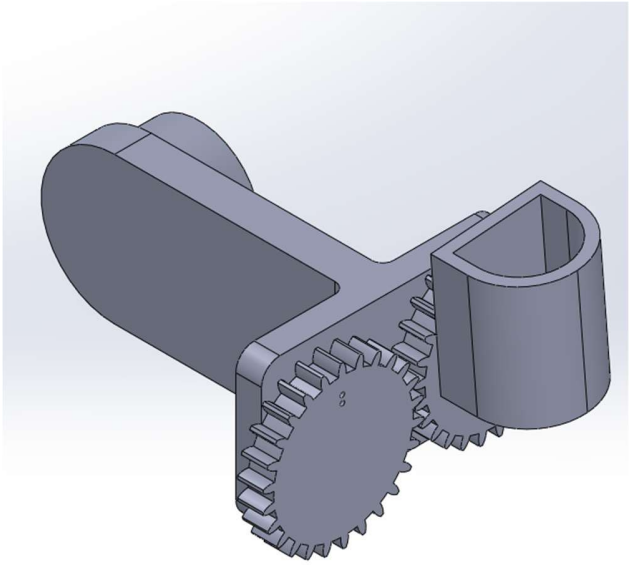


*Figure 4: Ladle end-effector*

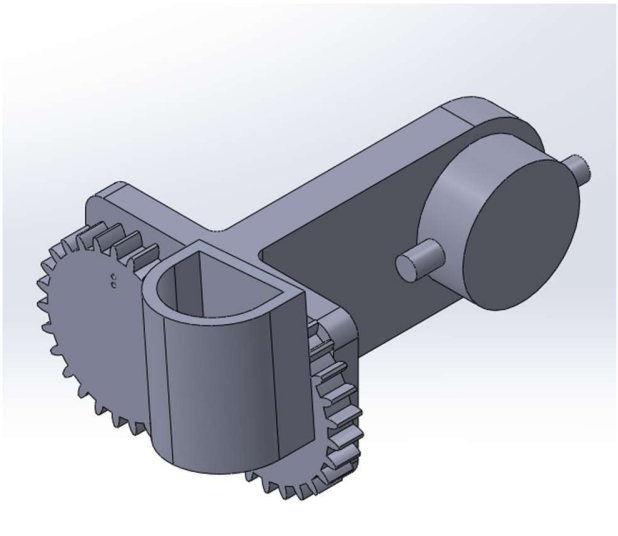


*Figure 5: Ladle end-effector with twist lock end*

### *4.1.7 Design Justification*

- **Choice of Arm Shape:**

A serial robotic arm configuration was chosen because it is simple to design, easy to control, and well-suited for replicating human arm motion. This makes it ideal for master–slave systems and teleoperation applications.

- **Why 5 DOF is Sufficient:**

Five degrees of freedom provide a good balance between complexity and functionality. While higher DOF systems offer more flexibility, they also increase cost and control complexity. For the intended tasks—such as object manipulation and pose replication—5 DOF is sufficient to achieve the required motion and orientation.

- **Joint Functionality:**

Each joint contributes to the overall movement of the arm:

- The base joint enables horizontal rotation
- The shoulder and elbow joints control vertical reach
- The wrist joints adjust the orientation of the end effector

Together, these joints allow smooth and coordinated motion, enabling the slave arm to accurately replicate the movements of the master arm.

## 4.2 Master Arm Mechanical Design

The master arm is designed to act as the input device for the system, where the user's motion is captured and translated into control signals for the slave arm. The mechanical design focuses on accurate angle measurement, stable sensor mounting, and smooth manual operation.

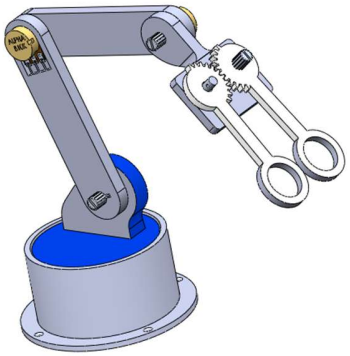


*Figure 6: Master Robot Arm*

### *4.2.1 Potentiometer Integration*

Each joint of the master arm is equipped with a rotary potentiometer to measure the angular position. The potentiometers are mechanically coupled to the joints, either directly through the joint shaft or via a linkage mechanism. This ensures that any movement of the arm is immediately reflected as a change in resistance, which is then converted into an electrical signal representing the joint angle.

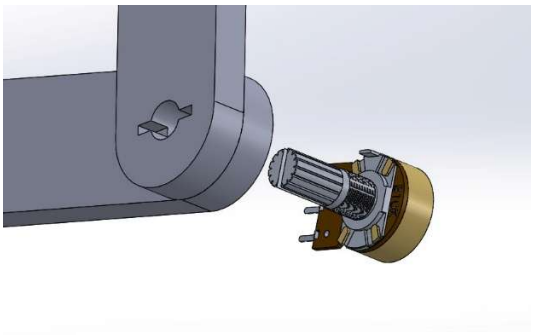


*Figure 7: Potentiometer fixation*

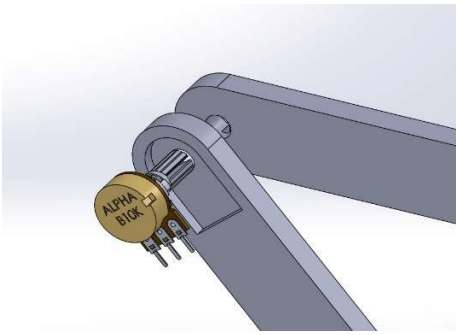



*Figure 8: Potentionmeter fixation from back*

### *4.2.2 Angle Calibration*

To ensure accurate mapping between the master and slave arms, a calibration process is performed. Each potentiometer is calibrated by defining its minimum and maximum angle limits corresponding to the physical range of motion of the joint. These values are then scaled to match the input range required by the control system, allowing consistent and precise angle transmission.

### *4.2.3 Fixation and Stability*

Proper fixation of the potentiometers is critical for reliable performance. The sensors are securely mounted using brackets or holders to prevent any relative movement between the potentiometer body and the arm structure. This eliminates measurement errors caused by slipping or misalignment. Additionally, the joints are designed to minimize mechanical play, ensuring smooth motion and accurate angle tracking.

### *4.2.4 End Effector Mechanism*

The master gripper is mechanically constructed as a two-finger input device to enable the operator to hand control the grip action of the slave arm. Finger gearing is made up of two gears with the same gear ratio that are meshed together, so that both fingers move in unison and in opposite directions during opening and closing. Each finger is equipped with a unique opening that enables the operator to insert and easily operate the mechanism with their own fingers at the end of each finger. The ergonomic design allows for more control accuracy and for intuitive human interaction with the master arm during teleoperation tasks.

## 4.3 Electrical and Electronics Design

The electrical and electronics subsystem forms the core of the control and communication architecture of the robotic system. It integrates sensing, processing, wireless transmission, and actuation to enable accurate and reliable operation of the master–slave robot arms.

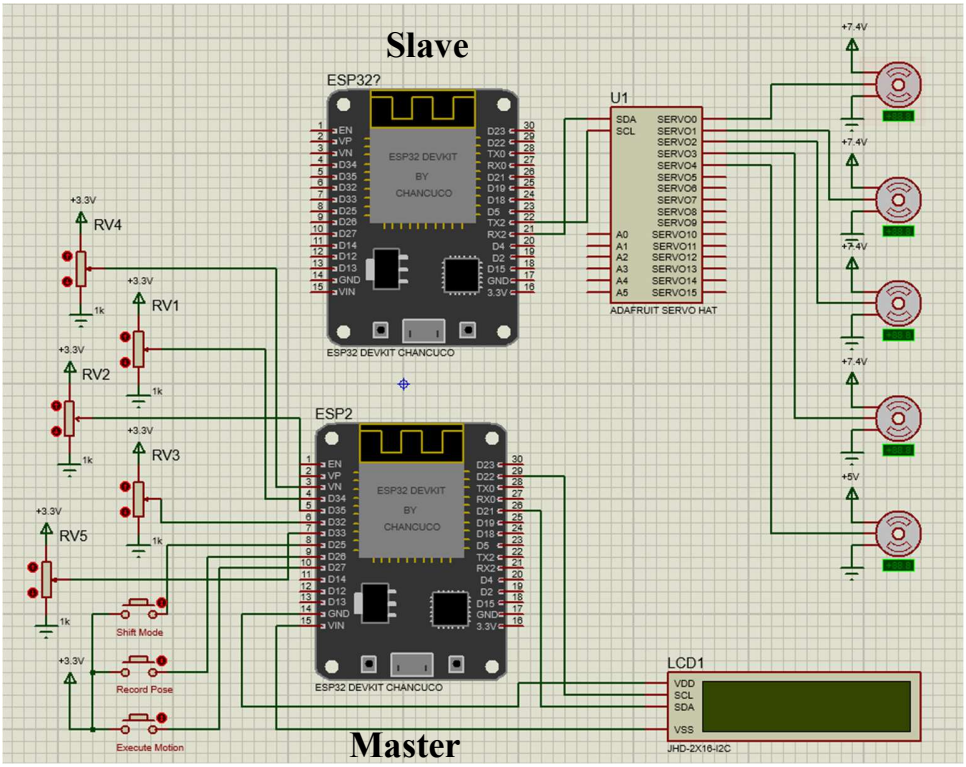



*Figure 9: Hardware diagram and connections*

The system is built around the ESP32 microcontroller, which serves as the main processing unit. It is responsible for reading input signals from the sensors, processing the data, handling wireless communication, and generating control signals for the actuators.

Position feedback from the master arm is obtained using potentiometers (pot), which are attached to each joint. These sensors convert mechanical motion into electrical signals, allowing the system to measure joint angles accurately. The analog signals from the potentiometers are read by the ESP32 and mapped into corresponding control commands.

Wireless communication between the master and slave systems is achieved using the ESP-NOW protocol, which is implemented on the ESP32. This method provides a fast

and efficient way to transmit data with low latency, making it suitable for real-time control applications.

Motor control on the slave side is handled using the PCA9685 module. This driver generates precise PWM signals required to control multiple servo motors simultaneously, ensuring smooth and synchronized motion of the robotic arm joints.

The system is powered by an 8.4V battery, which supplies the necessary energy for the microcontroller, motor drivers, and actuators. Proper voltage regulation is considered to ensure stable operation of all components.

User interaction is incorporated through push buttons, which are used to switch between different operating modes, such as real-time control and record-and-execute mode. Additionally, an LCD display is used to provide visual feedback to the user by indicating the current mode of operation and system status.

All components are interconnected using jumper wires, forming the complete circuit of the system. Careful wiring and organization are essential to ensure reliable connections and minimize signal noise.

In terms of functionality, the system operates as follows: the sensors (potentiometers) read the positions of the master arm joints, the ESP32 processes this data and converts it into control signals, the wireless communication module (ESP-NOW) transmits the data to the slave system, and finally, the motor driver (PCA9685) generates PWM signals to drive the motors of the slave arm, resulting in accurate motion replication.

### *4.3.1 Actuators*

The motion of the slave robotic arm is achieved using servo motors installed at each joint. For the main joints of the arm, FT7135M servo motors are used. These motors were selected due to their higher torque capability, which is necessary to support the load of the arm links and ensure stable and precise movement across all five degrees of freedom. Their performance allows the arm to operate smoothly while maintaining sufficient strength for handling different tasks.

For the end effector, specifically the gripper, a SG90 servo motor is used. This motor is lightweight and suitable for smaller load requirements, making it ideal for controlling the opening and closing mechanism of the gripper without adding unnecessary weight to the end of the arm.

The combination of high-torque FT7135M motors for the main joints and a compact SG90 motor for the gripper ensures an efficient balance between power, precision, and weight distribution. These motors are controlled via PWM signals generated by the PCA9685 module, allowing coordinated and accurate movement of all joints in the robo tic arm.




*Figure 10: FT7135M Metal Gear Servo (35 kg.cm)*




*Figure 11: SG90 Micro Servo 180 Degree*

### *4.3.2 Battery and Power Consumption*

**Battery Capacity:** The battery is rated at 1000 mAh, but due to losses, only about 80% is usable: Effective capacity=800 mA

**Motor Current Consumption**

The system uses:

- **4 FT7135M motors (main joints)**
- **1 SG90 motor (gripper)**

Each motor has 3 operating states:

- Idle (low current)
- Running (medium current)
- Stall (very high current)

$$Iavg = (\%idle \times Iidle) + (\%run \times Irun) + (\%stall \times Istall)$$

Example for one FT7135M motor:

$$= (0.3 \times 280) + (0.6 \times 680) + (0.1 \times 3000) = 792\ mA$$

- Total for 4 motors:

$$792 \times 4 = 3168 \text{ mA}$$

- SG90 motor:

$$\approx 137 \text{ mA}$$

**Total current consumption:**

$$I_{total} \approx 3305 \text{ mA}$$

**Operating Time Calculation**

$$\text{Time} = \frac{\text{Battery capacity}}{\text{Total current}} = \frac{800}{3305} \approx 0.24 \text{ hours} \approx 14.5 \text{ minutes}$$

The robotic arm operates for approximately:

$$\boxed{\text{14 to 15 minutes}}$$

# 5. Control Strategy

## 5.1 Control System / Software

### *5.1.1 Master Arm Control System*

#### 1. How the system reads angles

Each of the 5 joints has a potentiometer wired to an ADC pin. When you move the master arm, the resistance changes, and readStableADC() averages 32 ADC samples per pin (with 100µs settling time between each) to reduce noise. That raw 12-bit value (0–4095) is then mapped to a 0–180° angle — but each joint applies its own offset and scaling to match the physical range of that joint. For example, joint 5 applies (value - 150) × 2 to stretch a narrow pot range into full servo range.

#### 2. How data is transmitted wirelessly

The angles are packed into an ArmPosition struct — just 5 bytes, one per joint. This is sent via ESP-NOW, a connectionless Wi-Fi protocol that transmits directly by MAC address with no router or pairing handshake. The master simply calls esp_now_send() and the slave receives the same struct. In real-time mode, a send is triggered only when any joint changes by ≥3° (a deadband filter to avoid flooding the channel).

#### 3. How the slave reproduces motion

The slave ESP32 receives the same ArmPosition struct via its own ESP-NOW receive callback, then writes each joint[i] value directly to the corresponding servo as a position command. The slave is passive — it has no sensors of its own, it just obeys whatever the master sends.

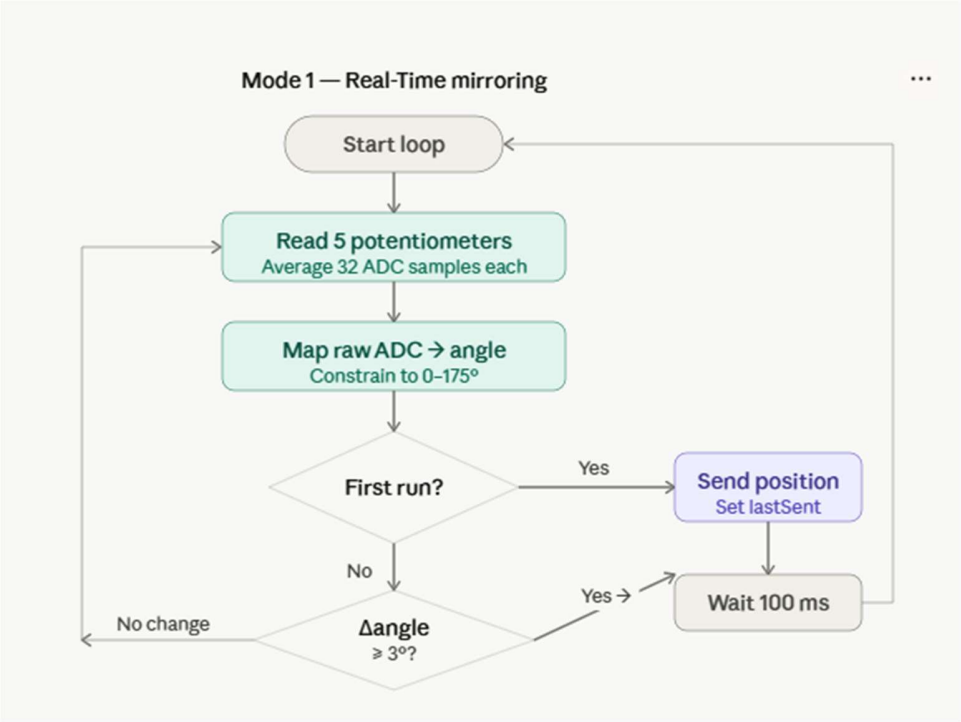



*Figure 12: mode 1 algorithm flowchart*

## 4. How record mode stores poses

When you press RECORD, readMasterPosition() takes a snapshot of all 5 joint angles and stores it in session.waypoints[count]. Up to 4 such snapshots can be stored (indices 0–3). The count increments after each save. No interpolation happens here — it's a raw freeze-frame of wherever the master arm is sitting at that moment.

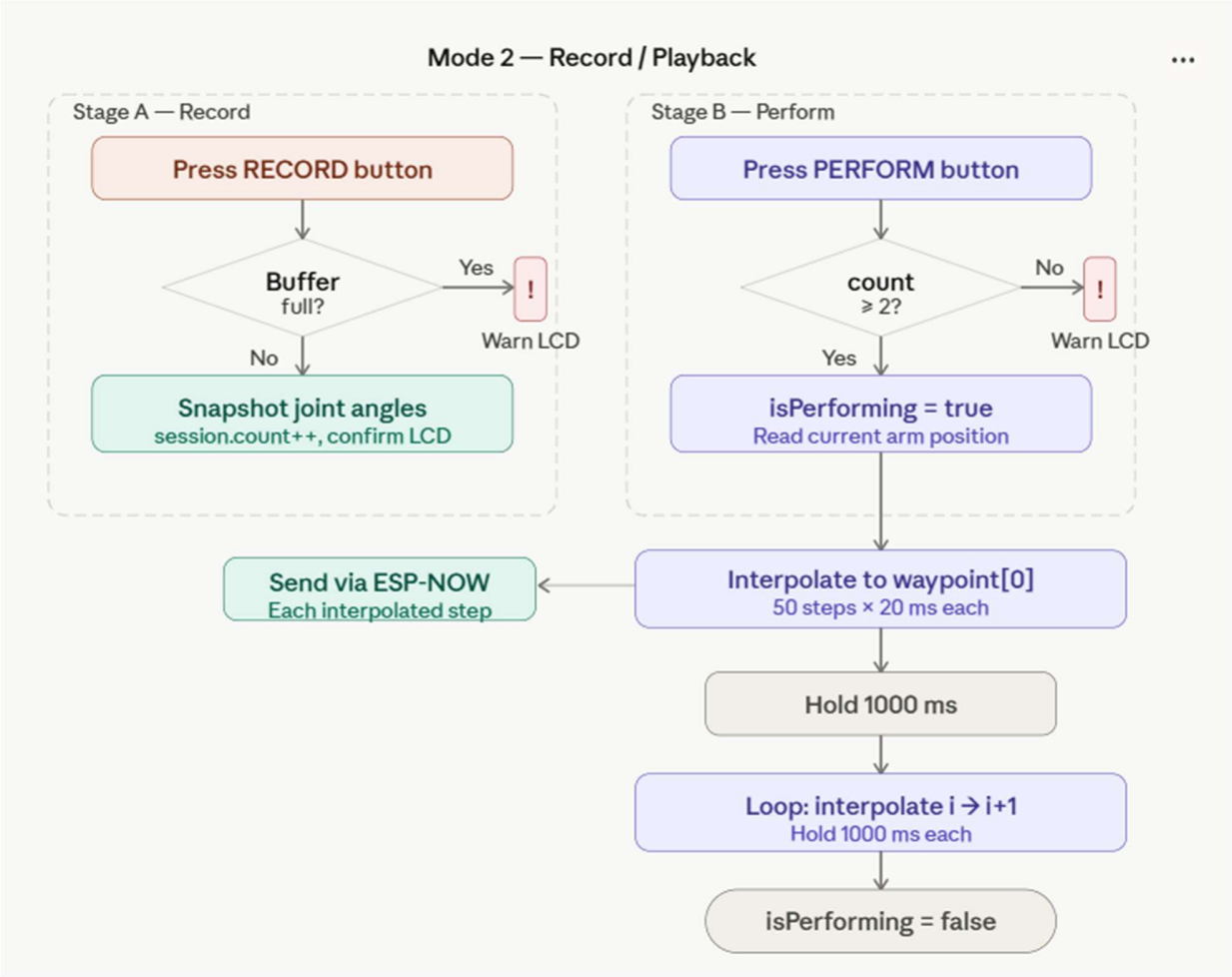



*Figure 13: record algorithm flowchart*

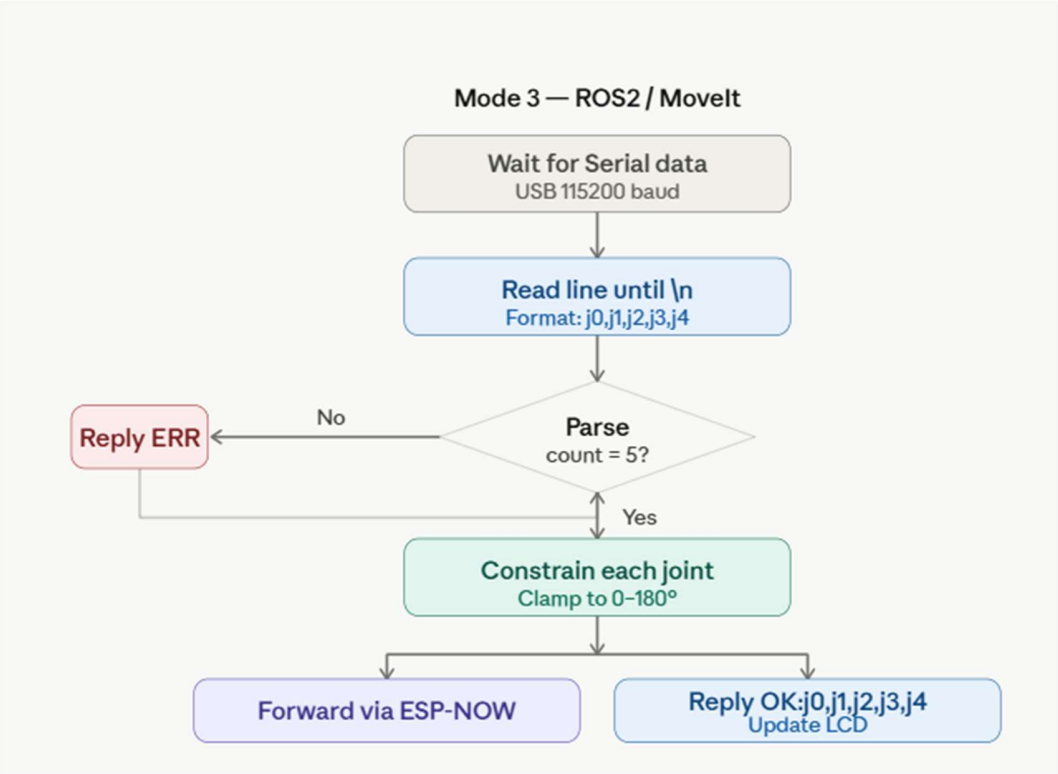



*Figure 14: Mode 3 flowchart diagram*

### *5.1.2 Slave Arm Control System*

The slave does exactly three things:

Receive — ESP-NOW fires onReceive() when a packet arrives. It copies the 5 joint angles into a volatile buffer and sets a flag. Nothing else — no hardware access allowed in interrupt context.

Copy safely — The main loop polls the flag every 10 ms. When set, it freezes interrupts for a moment, copies the buffer into a clean local variable, then re-enables interrupts. This prevents a mid-copy overwrite.

Drive servos — It converts each angle (0–180°) into a PWM tick count (102–492) and sends it to the PCA9685 over I²C. The PCA9685 generates the actual 50 Hz pulse that moves the servo.

That's the entire slave. It has no sensors, no decisions, no modes — just receive, copy, move.

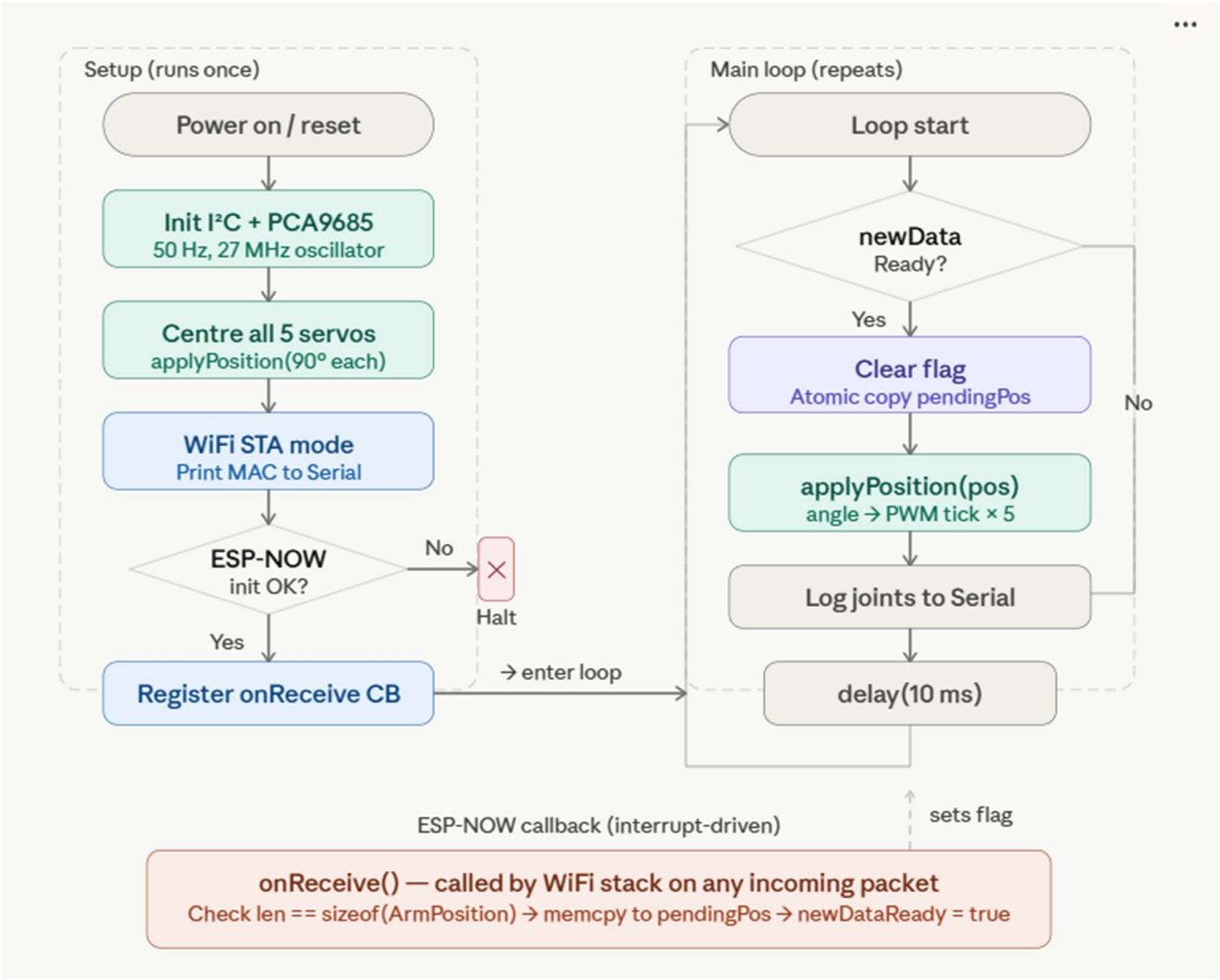



*Figure 15: Slave arm flowchart*

## 5.2 Communication Method

The communication system is a key component of the project, enabling real-time data transfer between the master and slave robotic arms.

### Wireless Communication Selection

Wireless communication was chosen to allow flexible and remote operation without physical constraints. This is especially important for applications in environments that are difficult or unsafe for human access.

### Type of Communication (ESP-NOW)

The system uses ESP-NOW, a connectionless peer-to-peer wireless communication protocol developed by Espressif Systems for ESP32 devices. ESP-NOW allows direct communication between devices without the need for a router or Wi-Fi network, making it highly suitable for low-latency and real-time applications.

In this setup, the master ESP32 acts as a transmitter, sending joint angle data, while the slave ESP32 acts as a receiver, processing the data and controlling the motors accordingly.

### Data Format

The transmitted data is organized into a structured packet containing the angular positions of the five joints. Each value is assigned a fixed position in the data frame, ensuring correct mapping between the master and slave arms. This structured format simplifies decoding and reduces processing time.

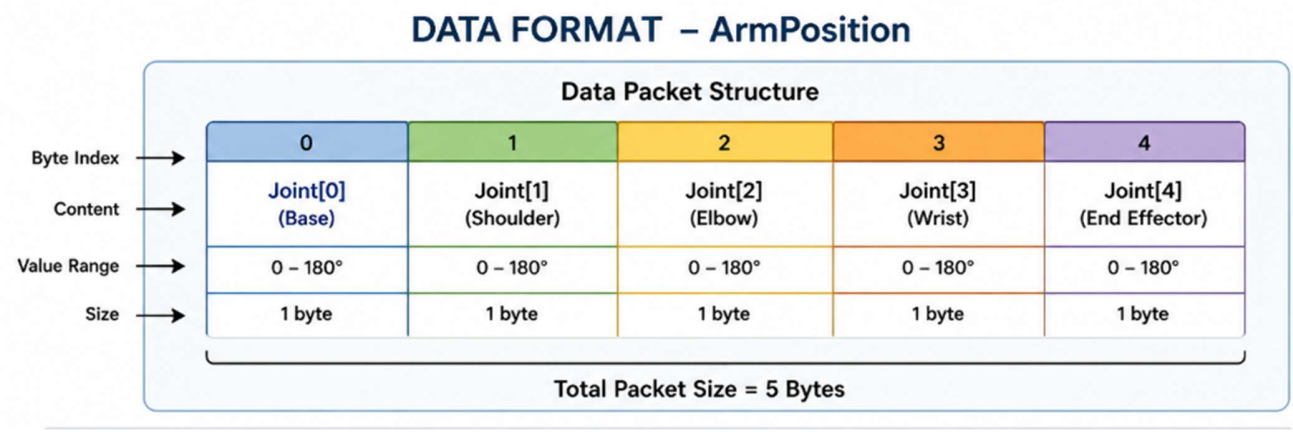



*Figure 16: data format*

**Delay and Latency**

ESP-NOW provides very low latency compared to traditional Wi-Fi communication, which is essential for real-time control. The system achieves fast response by transmitting small data packets at a high rate, allowing the slave arm to closely follow the master arm movements with minimal delay.

**Transmission Reliability**

To ensure reliable performance, the system implements:

- Continuous and high-frequency data transmission
- Use of ESP-NOW acknowledgment mechanisms to confirm successful delivery
- Simple validation of received data before execution
- Stable power supply to avoid communication interruptions

These features make ESP-NOW a robust and efficient solution for wireless robotic control, providing both speed and reliability.

## 5.3 Bonus Work: Third Mode (ROS2 and Motion Planning Integration)

As an advanced extension of the project, a third control mode is proposed to enable intelligent motion planning and semi-autonomous operation of the robotic arm using MoveIt within ROS 2. This mode represents a significant step beyond manual control, aiming to enhance the system with modern robotics capabilities.

The development process began by creating a digital model of the robot. The robot arm design was exported from SolidWorks as URDF and mesh files using a dedicated plugin tool. A new ROS2 workspace named “Jarvis” (the same name as the robot arm) was then created to organize the development environment. Inside this workspace, a package called “jarvis_description” was developed. This package contains the robot’s mesh files, along with the URDF model, which was later converted into a Xacro file to allow more flexible and modular configuration.

To visualize and simulate the robot, two main launch files were created: *jarvis.display.py* for visualization in RViz and *jarvis.gazebo.py* for simulation in Gazebo. These tools allow testing the robot model in a virtual environment before applying commands to the real hardware, reducing risks and improving development efficiency.

The next step involved integrating the robot with MoveIt to generate motion planning and control algorithms for both the robotic arm and the end effector. This stage presented several technical challenges, including configuration errors and controller mismatches. Multiple iterations and code modifications were required to resolve these issues, and eventually a functional motion planning system was achieved.

The final goal of this mode is to bridge the gap between simulation and real-world execution by integrating MoveIt with the physical robot (via microcontroller control). This allows planned trajectories generated in MoveIt to be executed by the actual robotic arm, enabling precise and optimized movements.

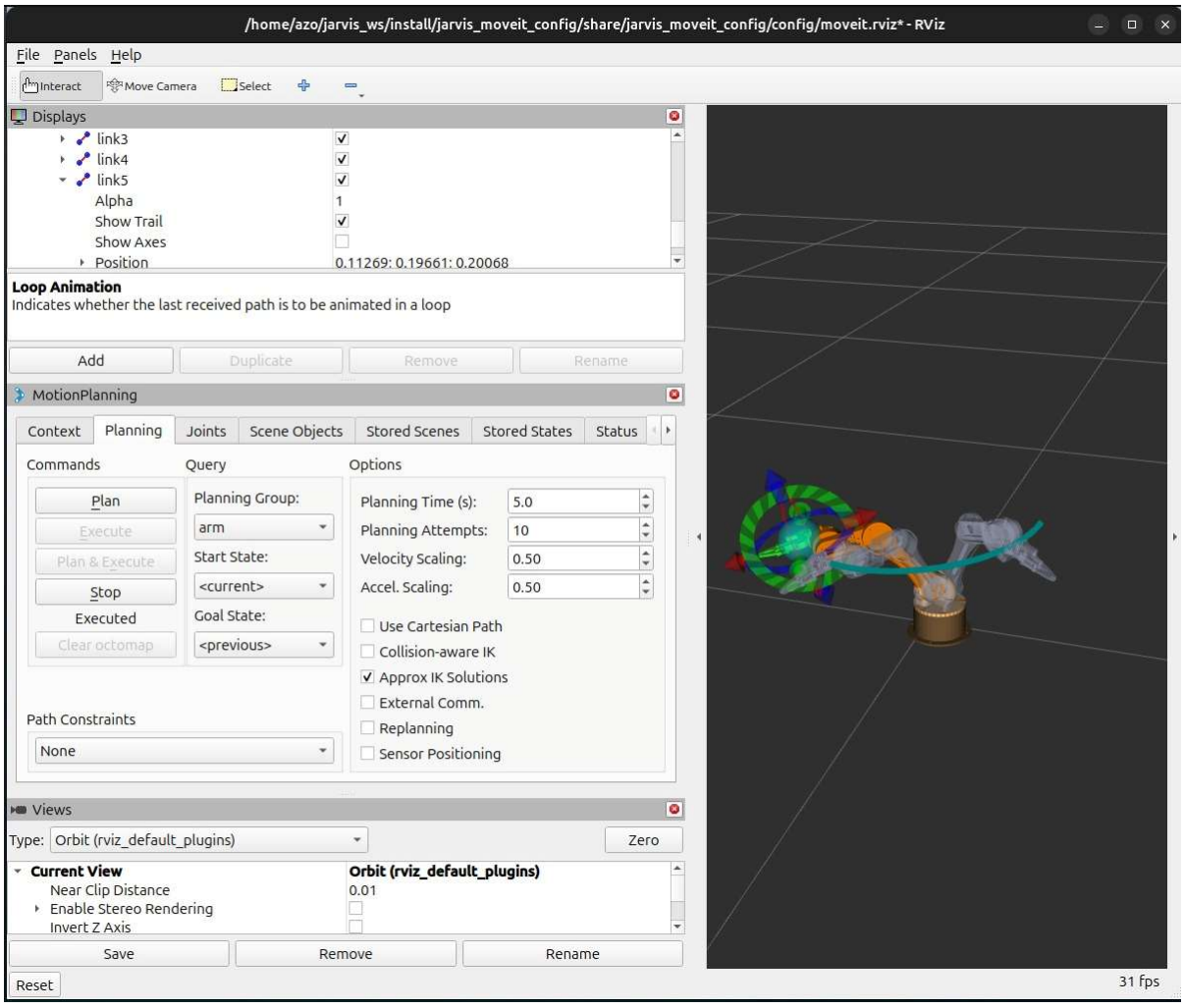



*Figure 17: Rviz & Moveit planning and executing*

## 5.4 Robot Arm Control System Using ROS 2 & MoveIt 2

This system implements a 5-DOF robotic arm controlled through the ROS 2 ecosystem, leveraging MoveIt 2 for motion planning and an Arduino-based hardware interface for physical actuation. The architecture spans from CAD design to real servo motor movement across two tightly integrated layers.

### *5.4.1 Layer 1: Software Configuration & Launch Infrastructure*

The pipeline originates in SolidWorks, where the robot is designed and exported via the sw2urdf plugin into a structured URDF package (jarvis_description/urdf/), containing three key files:

- jarvis.urdf.xacro — the main robot model
- ros2_control.xacro — hardware interface definitions
- gazebo.xacro — simulation plugins

These feed into the MoveIt configuration package (jarvis_moveit_config), which holds all planning parameters including joint limits, kinematics (KDL IK solver), OMPL planner settings (RRTConnect), sensor configurations, and initial joint positions. The launch layer then orchestrates four simultaneous components — move_group, moveit_rviz, spawn_controllers, and Gazebo simulation — ultimately bringing the system to a MoveIt-ready state, exposing a FollowJointTrajectory action interface.

### *5.4.2 Layer 2: Trajectory Execution & Hardware Control*

Once MoveIt 2 generates a motion plan, the execution flow proceeds as follows:

- OMPL Planner (RRTConnect) computes a collision-free joint trajectory
- Trajectory Optimizer applies time-optimal parameterization
- Controller Manager (simple_controller_manager) dispatches the trajectory via the FollowJointTrajectory action to a custom ROS 2 node

The arduino_controller.py node acts as the bridge between ROS 2 and physical hardware. It receives the goal trajectory, converts joint angles from radians to servo degrees, clamps all values within 0–180° with offset correction for joints j1–j5, then transmits them over Serial (115200 baud) to the Arduino.

On the hardware side, the Arduino receives the joint angle string, drives a PCA9685 PWM servo shield (mapping 0–180° to pulse widths of 150–600), and actuates 5 physical servo motors.

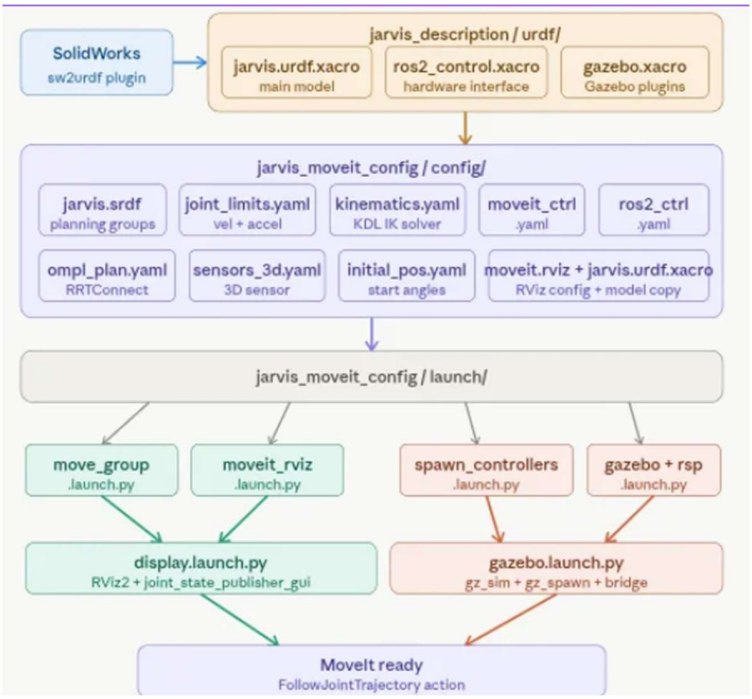



*Figure 18: configuration  diagram of mode 3*

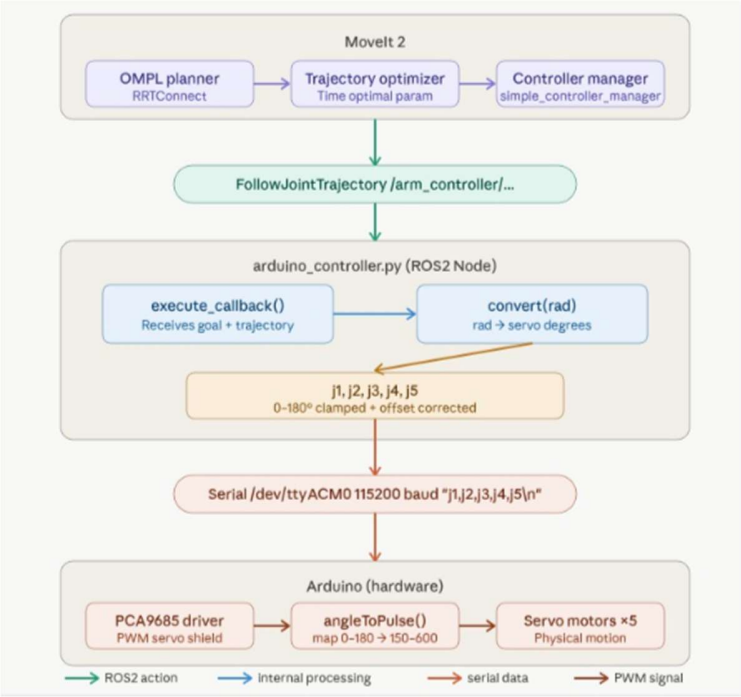



*Figure 19: control diagram of mode 3*

# 6. Conclusion

The wireless master–slave robotic arm system was successfully designed and implemented as an integrated mechatronics project combining mechanical design, embedded systems, wireless communication, and robotic control. The project achieved accurate real-time motion replication between the master and slave arms using ESP32 microcontrollers and the ESP-NOW communication protocol, providing low-latency and reliable wireless operation.

The mechanical design of the 5-DOF robotic arm demonstrated a balance between functionality, simplicity, and cost-effectiveness, while the modular end-effector mechanism increased the flexibility of the system by allowing different tools to be attached depending on the application requirements. In addition, the implementation of real-time control and record-and-execute modes enabled both direct teleoperation and basic task automation.

The project also demonstrated the integration of electronic control, sensing systems, servo actuation, and power management into a complete robotic platform. Furthermore, the ROS2 and MoveIt integration introduced advanced motion planning and simulation capabilities, extending the project toward intelligent and semi-autonomous robotic applications.

Overall, the project represents a practical and scalable robotic manipulation system that can be further developed for industrial automation, remote handling, educational purposes, and research applications. Future improvements may include computer vision integration, closed-loop feedback systems, trajectory interpolation, and AI-based autonomous control to further enhance the performance and capabilities of the robotic arm.

# Appendices

The master code:

```
#include <esp_now.h>
#include <WiFi.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

// ── Pin Definitions ──────────────────────────────────────────
const int POT_PINS[]    = {32, 33, 34, 35, 39};
const int NUM_JOINTS    = 5;

const int SHIFT_BTN     = 25;   // toggles mode 1 ↔ mode 2
const int RECORD_BTN    = 26;   // (mode 2) save current position
const int PERFORM_BTN   = 27;   // (mode 2) replay recorded positions

// ── ESP-NOW ──────────────────────────────────────────────────
uint8_t RECEIVER_MAC[] = {0x44, 0x1D, 0x64,  0xBD, 0x7E, 0xA8};

// ── Constants ────────────────────────────────────────────────
const int   MAX_POSITIONS       = 5;      // max storable waypoints
const int   POT_RAW_MIN         = 683;    // ~1/6 of 4095 (0.55 V)
const int   POT_RAW_MAX         = 3413;   // ~5/6 of 4095 (2.75 V)
const int   INTERP_STEPS        = 50;     // smoothing steps between waypoints
const int   INTERP_STEP_DELAY   = 20;     // ms between each interpolation step
const int   HOLD_DELAY          = 1000;   // ms to hold at each waypoint
const int   REALTIME_DELAY      = 100;    // ms between real-time sends
const int   Min_angle           = 3;      // degree (angle)
// ── Data Structures ──────────────────────────────────────────

/** Packet sent to slave via ESP-NOW. */
typedef struct {
  uint8_t joint[5];   // angles 0–180° for joints 1–5
} ArmPosition;

/** All state for mode-2 recording/playback. */
typedef struct {
  ArmPosition waypoints[MAX_POSITIONS];  // recorded snapshots
  int         count;                     // number of waypoints saved so far
  bool        isPerforming;              // true while replaying
} RecordSession;

// ── Globals ──────────────────────────────────────────────────
LiquidCrystal_I2C lcd(0x27, 16, 2);   // adjust address if needed (0x3F)
```

```
esp_now_peer_info_t peerInfo;

RecordSession session = { .count = 0, .isPerforming = false };

enum OperatingMode { MODE_REALTIME = 1, MODE_RECORD = 2, MODE_ROS2 = 3 };
OperatingMode currentMode = MODE_REALTIME;

// Debounce timestamps (ms)
unsigned long lastShiftPress   = 0;
unsigned long lastRecordPress  = 0;
unsigned long lastPerformPress = 0;
const unsigned long DEBOUNCE_MS = 300;

// ─── ESP-NOW Callback ──────────────────────────────────────
void onSent(const uint8_t *mac, esp_now_send_status_t status) {
  // Uncomment for verbose debug:
  // Serial.println(status == ESP_NOW_SEND_SUCCESS ? "Sent OK" : "Send FAILED");
}

// ─── Helper: Read potentiometers → ArmPosition ─────────────
/**
 * Reads all 5 potentiometers and maps raw ADC values to 0–180°.
 * @return  ArmPosition struct with current joint angles.
 */
int readStableADC(int pin, int samples = 32) {
  long sum = 0;
  for (int i = 0; i < samples; i++) {
    sum += analogRead(pin);
    delayMicroseconds(100); // let ADC settle between reads
  }
  return sum / samples;
}

ArmPosition readMasterPosition() {
  ArmPosition pos;

  int raw = readStableADC(POT_PINS[0], 32); // 0 ~ 4095
  pos.joint[0] = (uint8_t) constrain(
    map(raw, 0, 4095, 0, 270), 0, 175
  );
  raw = readStableADC(POT_PINS[1], 32);
  pos.joint[1] = 180 - (uint8_t) constrain(
    (map(raw, 0, 4095, 0, 270) - 55), 0, 175
  );
  raw = readStableADC(POT_PINS[2], 32);
```

```
  pos.joint[2] = (uint8_t) constrain(
    (map(raw, 0, 4095, 0, 270) + 10), 0, 175
  );
  raw = readStableADC(POT_PINS[3], 32);
  pos.joint[3] = (uint8_t) constrain(
    map(raw, 0, 4095, 0, 270) - 80, 0, 175
  );
  raw = readStableADC(POT_PINS[4], 32);
  pos.joint[4] = (uint8_t) constrain(
    (map(raw, 0, 4095, 0, 270) - 150)*1.2, 0, 175
  );

  return pos;
}

// ── Helper: Send ArmPosition via ESP-NOW ──────────────────
/**
 * Transmits an ArmPosition struct to the slave ESP32.
 * @param pos  Position to send.
 */
void sendPosition(const ArmPosition &pos) {
  esp_now_send(RECEIVER_MAC, (uint8_t *)&pos, sizeof(ArmPosition));
}

// ── Helper: Interpolate & send between two waypoints ──────
/**
 * Smoothly moves from `from` to `to` in INTERP_STEPS steps,
 * sending each intermediate position to the slave.
 *
 * @param from  Starting joint angles.
 * @param to    Target joint angles.
 */
void interpolateTo(const ArmPosition &from, const ArmPosition &to) {
  for (int step = 1; step <= INTERP_STEPS; step++) {
    ArmPosition mid;
    for (int j = 0; j < NUM_JOINTS; j++) {
      // Linear interpolation: from + (to - from) * (step / INTERP_STEPS)
      mid.joint[j] = (uint8_t)(
  from.joint[j] + (float)(to.joint[j] - from.joint[j]) * step / INTERP_STEPS
      );
    }
    sendPosition(mid);
    delay(INTERP_STEP_DELAY);
  }
}
```

```
// ── LCD Helpers ──────────────────────────────────────────
/** Clear LCD and print two rows. Pass "" for an empty row. */
void lcdPrint(const char *row0, const char *row1) {
  lcd.clear();
  lcd.setCursor(0, 0); lcd.print(row0);
  lcd.setCursor(0, 1); lcd.print(row1);
}

/** Update LCD to reflect current mode and record session state. */
void updateLCD() {
  if (currentMode == MODE_REALTIME) {
    lcdPrint("Mode 1: RealTime", "Mirroring...    ");
  } else if (currentMode == MODE_RECORD) {
    // Mode 2 — show stage
    if (session.isPerforming) {
      lcdPrint("Mode 2: Perform ", "Playing back... ");
    } else {
      char row1[17];
      snprintf(row1, sizeof(row1), "Pts: %d/%d  REC",
               session.count, MAX_POSITIONS);
      lcdPrint("Mode 2: Record  ", row1);
    }
  } else {
    // Mode 3 — ROS2 / MoveIt
    lcdPrint("Mode 3: ROS2    ", "Waiting...      ");
  }
}

// ── Button: SHIFT ────────────────────────────────────────
/**
 * Toggles between MODE_REALTIME and MODE_RECORD.
 * Resets the recording session when entering Mode 2.
 */
void handleShiftButton() {
  unsigned long now = millis();
  if (digitalRead(SHIFT_BTN) == LOW && (now - lastShiftPress) > DEBOUNCE_MS) {
    lastShiftPress = now;

    if (currentMode == MODE_REALTIME) {
      // 1 → 2
      currentMode = MODE_RECORD;
      session.count       = 0;
      session.isPerforming = false;
      Serial.println("[SHIFT] → Mode 2: Record/Playback");
    } else if (currentMode == MODE_RECORD) {
      // 2 → 3
```

```
      currentMode = MODE_ROS2;
      session.isPerforming = false;
      Serial.println("[SHIFT] → Mode 3: ROS2/MoveIt");
      Serial.println("[ROS2] Ready. Send: j0,j1,j2,j3,j4<LF>");
    } else {
      // 3 → 1
      currentMode = MODE_REALTIME;
      Serial.println("[SHIFT] → Mode 1: Real-Time");
    }
    updateLCD();
  }
}

// ─── Button: RECORD (Mode 2, Stage A) ──────────────────────
/**
 * Saves the current master arm position as the next waypoint.
 * Ignored if the session is full or playback is running.
 */
void handleRecordButton() {
  unsigned long now = millis();
  if (digitalRead(RECORD_BTN) == LOW && (now - lastRecordPress) > DEBOUNCE_MS) {
    lastRecordPress = now;

    if (currentMode != MODE_RECORD || session.isPerforming) return;

    if (session.count >= MAX_POSITIONS) {
      Serial.println("[RECORD] Buffer full!");
      lcdPrint("Mode 2: Record  ", "Buffer FULL!    ");
      delay(1000);
      updateLCD();
      return;
    }

    // Snapshot current position
    session.waypoints[session.count] = readMasterPosition();
    Serial.printf("[RECORD] Saved position %d: J1=%d J2=%d J3=%d J4=%d J5=%d\n",
      session.count + 1,
      session.waypoints[session.count].joint[0],
      session.waypoints[session.count].joint[1],
      session.waypoints[session.count].joint[2],
      session.waypoints[session.count].joint[3],
      session.waypoints[session.count].joint[4]);
    session.count++;

    // Brief confirmation on LCD
    char msg[17];
```

```
    snprintf(msg, sizeof(msg), "Saved %d/%d!     ", session.count, MAX_POSITIONS);
    lcd.setCursor(0, 1);
    lcd.print(msg);
    delay(500);
    updateLCD();
  }
}

// ─── Button: PERFORM (Mode 2, Stage B) ──────────────────────
/**
 * Replays all recorded waypoints in sequence with smooth interpolation.
 * Requires at least 2 waypoints to interpolate.
 */
void handlePerformButton() {
  unsigned long now = millis();
  if (digitalRead(PERFORM_BTN) == LOW && (now - lastPerformPress) > DEBOUNCE_MS) {
    lastPerformPress = now;

    if (currentMode != MODE_RECORD || session.isPerforming) return;

    if (session.count < 2) {
      Serial.println("[PERFORM] Need at least 2 positions.");
      lcdPrint("Mode 2: Perform ", "Need >= 2 pts!  ");
      delay(1500);
      updateLCD();
      return;
    }

    // ── Begin playback ──
    session.isPerforming = true;
    updateLCD();
    Serial.printf("[PERFORM] Replaying %d positions...\n", session.count);

    // Move to the first waypoint from wherever the arm currently is
    ArmPosition current = readMasterPosition();
    interpolateTo(current, session.waypoints[0]);
    delay(HOLD_DELAY);

    // Traverse through all recorded waypoints
    for (int i = 0; i < session.count - 1; i++) {
      Serial.printf("[PERFORM] Moving: waypoint %d → %d\n", i + 1, i + 2);
      interpolateTo(session.waypoints[i], session.waypoints[i + 1]);
      delay(HOLD_DELAY);     // pause at each waypoint
    }

    // Hold at the final position
```

```
    sendPosition(session.waypoints[session.count - 1]);
    Serial.println("[PERFORM] Playback complete.");

    session.isPerforming = false;
    updateLCD();
  }
}

// ─── Mode 1: Real-Time Loop ──────────────────────────────────
/** Reads master arm and immediately forwards position to slave. */
void runRealTimeMode() {
  static ArmPosition lastSent = {};
  static bool initialized = false;

  ArmPosition pos = readMasterPosition();

  if (!initialized) {
    sendPosition(pos);
    lastSent = pos;
    initialized = true;
    delay(REALTIME_DELAY);
    return;
  }

  bool changed = false;
  for (int i = 0; i < NUM_JOINTS; i++) {
    if (abs((int)pos.joint[i] - (int)lastSent.joint[i]) >= Min_angle) { // 2° threshold
      changed = true;
      break;
    }
  }

  if (changed) {
    sendPosition(pos);
    lastSent = pos;
    Serial.printf("[RT] J1=%d J2=%d J3=%d J4=%d J5=%d\n",
      pos.joint[0], pos.joint[1], pos.joint[2], pos.joint[3], pos.joint[4]);
  }
  delay(REALTIME_DELAY);
}

// ─── Mode 3: ROS2 / MoveIt Serial Loop ──────────────────────
/**
 * Listens for joint-angle packets arriving from a ROS2/MoveIt node
 * over USB Serial in the format:
 *
```

```
 *   "j0,j1,j2,j3,j4\n"   (5 integers, 0–180 each)
 *
 * On success  → forwards the position via ESP-NOW and replies "OK:j0,j1,j2,j3,j4\n"
 * On failure  → replies "ERR\n"
 *
 * The ROS2 serial_bridge node should open the same port at 115200 baud
 * and publish/subscribe on the /arm_joint_states topic.
 */
void runROS2Mode() {
  if (!Serial.available()) return;

  String data = Serial.readStringUntil('\n');
  data.trim();

  // ── Parse ──────────────────────────────────────────────
  int j[5];
  int count = sscanf(data.c_str(), "%d,%d,%d,%d,%d",
                     &j[0], &j[1], &j[2], &j[3], &j[4]);

  if (count != 5) {
    Serial.println("ERR");
    return;
  }

  // ── Validate & build packet ────────────────────────────
  ArmPosition pos;
  for (int i = 0; i < NUM_JOINTS; i++) {
    pos.joint[i] = (uint8_t) constrain(j[i], 0, 180);
  }

  // ── Forward via ESP-NOW to slave ───────────────────────
  sendPosition(pos);

  // ── Acknowledge to ROS2 node ───────────────────────────
  Serial.printf("OK:%d,%d,%d,%d,%d\n",
                pos.joint[0], pos.joint[1], pos.joint[2],
                pos.joint[3], pos.joint[4]);

  // ── Update LCD with live joint values ──────────────────
  char row1[17];
  snprintf(row1, sizeof(row1), "%3d%3d%3d%3d%3d",
           pos.joint[0], pos.joint[1], pos.joint[2],
           pos.joint[3], pos.joint[4]);
  lcdPrint("Mode 3: ROS2    ", row1);
}
```

```
// ═══════════════════════════════════════════════════════
//  SETUP
// ═══════════════════════════════════════════════════════
void setup() {
  Serial.begin(115200);

  // ── ADC ──
  analogReadResolution(12);
  analogSetAttenuation(ADC_11db);

  // ── Buttons ──
  pinMode(SHIFT_BTN,   INPUT_PULLUP);
  pinMode(RECORD_BTN,  INPUT_PULLUP);
  pinMode(PERFORM_BTN, INPUT_PULLUP);

  // ── LCD ──
  lcd.init();
  lcd.backlight();
  lcdPrint("Robot Arm v2.0  ", "Initializing... ");

  // ── WiFi + ESP-NOW ──
  WiFi.mode(WIFI_STA);
  if (esp_now_init() != ESP_OK) {
    Serial.println("ESP-NOW init FAILED");
    lcdPrint("ESP-NOW FAILED  ", "Check MAC addr  ");
    while (true) delay(1000);
  }
  esp_now_register_send_cb(onSent);

  memcpy(peerInfo.peer_addr, RECEIVER_MAC, 6);
  peerInfo.channel = 0;
  peerInfo.encrypt = false;

  if (esp_now_add_peer(&peerInfo) != ESP_OK) {
    Serial.println("Failed to add peer");
    lcdPrint("Peer add FAILED ", "Check receiver  ");
    while (true) delay(1000);
  }

  Serial.println("Master controller ready.");
  updateLCD();   // show Mode 1 on LCD
}
```

```
void loop() {
  // Always check the mode-shift button
  handleShiftButton();

  if (currentMode == MODE_REALTIME) {
    // ── Mode 1: stream master → slave continuously ──
    runRealTimeMode();

  } else if (currentMode == MODE_RECORD) {
    // ── Mode 2: handle record & perform buttons ──
    if (!session.isPerforming) {
      // Only poll buttons when not mid-playback
      handleRecordButton();
      handlePerformButton();
    }
    // No delay here — playback is blocking inside handlePerformButton

  } else {
    // ── Mode 3: receive from ROS2/MoveIt and forward via ESP-NOW ──
    runROS2Mode();
  }
}
```

## Slave Code

```
#include <esp_now.h>
#include <WiFi.h>
#include <Wire.h>
#include <Adafruit_PWMServoDriver.h>

// ─── PCA9685 Setup ──────────────────────────────────────────
Adafruit_PWMServoDriver pwm = Adafruit_PWMServoDriver(0x40); // default I²C addr

// Servo PWM pulse range — tune these to match your servos
// Standard servos: ~500 µs (0°) to ~2400 µs (180°)
// At 50 Hz, one period = 20,000 µs → 4096 ticks
// tick = µs * 4096 / 20000
const int SERVO_MIN_TICK = 102;   // ≈ 500 µs  → 0°
const int SERVO_MAX_TICK = 492;   // ≈ 2400 µs → 180°
const int SERVO_FREQ_HZ  = 50;    // standard servo PWM frequency
```

```
const int NUM_JOINTS     = 5;     // must match master

// ── ESP-NOW ──────────────────────────────────────────────
// Must match the master's ArmPosition struct exactly
typedef struct {
  uint8_t joint[5];   // angles 0–180° for joints 1–5
} ArmPosition;

// ── State ────────────────────────────────────────────────
volatile bool      newDataReady = false;
volatile ArmPosition pendingPos  = {};
ArmPosition        currentPos   = {};

// ── Helper: angle (0–180°) → PCA9685 tick count ──────────
/**
 * Maps a joint angle to the correct PWM tick value for the PCA9685.
 * @param angle  Desired angle in degrees (0–180).
 * @return       Tick count to pass to setPWM().
 */
uint16_t angleToPwmTick(uint8_t angle) {
  return (uint16_t) map(angle, 0, 180, SERVO_MIN_TICK, SERVO_MAX_TICK);
}

// ── Helper: move one servo channel ───────────────────────
/**
 * Commands a single PCA9685 channel to the given angle.
 * @param channel  PCA9685 output channel (0–4 for joints 1–5).
 * @param angle    Target angle in degrees (0–180).
 */
void setServoAngle(uint8_t channel, uint8_t angle) {
  uint16_t tick = angleToPwmTick(constrain(angle, 0, 180));
  pwm.setPWM(channel, 0, tick);
}

// ── Helper: apply a full ArmPosition to all servos ───────
/**
 * Writes all 5 joint angles from an ArmPosition struct to the PCA9685.
 * @param pos  Position to apply.
 */
void applyPosition(const ArmPosition &pos) {
  for (int i = 0; i < NUM_JOINTS; i++) {
    setServoAngle(i, pos.joint[i]);
  }
}
```

```
void onReceive(const esp_now_recv_info_t *info, const uint8_t *data, int len) {
  if (len != sizeof(ArmPosition)) return;
  memcpy((void *)&pendingPos, data, sizeof(ArmPosition));
  newDataReady = true;
}
// ═══════════════════════════════════════════════════════
//  SETUP
// ═══════════════════════════════════════════════════════
void setup() {
  Serial.begin(115200);
  Serial.println("Slave arm booting...");

  // ── I²C + PCA9685 ──
  Wire.begin(21, 22);           // SDA=21, SCL=22
  pwm.begin();
  pwm.setOscillatorFrequency(27000000);  // calibrate to your board (typ. 25–27 MHz)
  pwm.setPWMFreq(SERVO_FREQ_HZ);
  delay(10);

  // Centre all servos at startup (90°)
  for (int i = 0; i < NUM_JOINTS; i++) {
    currentPos.joint[i] = 90;
  }
  applyPosition(currentPos);
  Serial.println("Servos centred at 90°.");

  // ── WiFi + ESP-NOW ──
  WiFi.mode(WIFI_STA);
  WiFi.disconnect();            // ensure not associated to any AP

  // Print this board's MAC so you can paste it into the master
  Serial.print("Slave MAC address: ");
  Serial.println(WiFi.macAddress());

  if (esp_now_init() != ESP_OK) {
    Serial.println("ESP-NOW init FAILED — halting.");
    while (true) delay(1000);
  }

  esp_now_register_recv_cb(onReceive);
  Serial.println("Slave ready — waiting for master packets.");
}

// ═══════════════════════════════════════════════════════
//  MAIN LOOP
// ═══════════════════════════════════════════════════════
```

```
void loop() {
  // Check if the ESP-NOW callback deposited a new position
  if (newDataReady) {
    newDataReady = false;

    // Copy from volatile buffer under atomic-safe window
    ArmPosition pos;
    noInterrupts();
    memcpy(&pos, (const void *)&pendingPos, sizeof(ArmPosition));
    interrupts();

    // Apply to all servos
    applyPosition(pos);
    currentPos = pos;

    Serial.printf("[RX] J1=%d J2=%d J3=%d J4=%d J5=%d\n",
      pos.joint[0], pos.joint[1], pos.joint[2],
      pos.joint[3], pos.joint[4]);
  }

  // Nothing else to do — servo commands are fire-and-forget via I²C.
  // Add a small yield so the ESP32 WiFi/ESP-NOW stack can run.
  delay(10);
}
```